# Supervised Fine-Tuning (SFT)

Teaching LLMs to Formulate Text & Follow Instructions

Yakov (Dima)

September 2026

# Two Purposes of SFT

**Alignment SFT**

Base completion model → general instruction-following assistant.

Teaches the model to act on instructions, not just continue text. Foundational, one-time.

**Domain / Application SFT**

Instruction-tuned model → specialized behavior for one product.

Already follows instructions — this adds *your* schema, vocabulary, and edge cases.

Same mechanism (masked next-token loss on instruction data) — just a different starting checkpoint and goal.

# Alignment SFT: Pre-training vs. SFT (The Mindset Shift)

### Base LLM — Text Completer

**Prompt:** Write a python function for quicksort.

**Completion:** Write a python function for mergesort.
Write a python function for binary search.
Write a python function for...

*Chaos pattern: keeps completing the list instead of answering it.*

### SFT LLM — Helpful Assistant

**Prompt:** Write a python function for quicksort.

**Completion:**

```
def quicksort(arr):  ...  return arr
```

*Alignment pattern: playing the persona of an assistant.*

# Domain SFT: An Autonomous-Driving Copilot

**General Instruct Model (Alignment SFT only)**

**Prompt:** Pedestrian at crosswalk distracted; truck in bike lane 20m ahead.

**Completion:** Moderate risk — distracted pedestrian near the crosswalk, truck blocking the bike lane ahead. Recommend slowing down.

*Limitation: correct but free-form prose — not parseable by a planner.*

**Domain-SFT'd Driving Copilot**

**Prompt:** (same)

**Completion:**

```
{"actors": [{"type":
"VRU_pedestrian", "risk":
"high"}, {"type":
"static_obstacle", "risk":
"medium"}], "action": "reduce
speed; widen clearance"}
```

*Result: fine-tuned on the AV stack's own logs.*

Domain SFT only had to teach *this* team's risk taxonomy and output format.

# Training: Same Loss, Adapted Rules

```
<|user|> What is 2+2?  <|assistant|> The answer is 4.
```

$$\mathcal{L} = -\sum_{t=i}^{N} \log P(x_t \mid x_{<t})$$

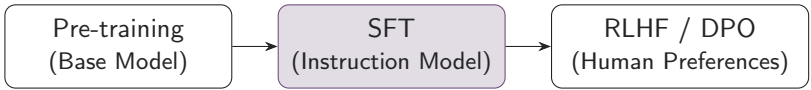
**No new objective:** same cross-entropy next-token prediction as pre-training.

**Masking:** loss computed only on assistant tokens.

**Chat template tokens** mark where turns begin/end, for masking and stopping.

**Learning rate:** dropped 1–2 orders of magnitude vs. pre-training.

# The LLM Alignment Pipeline




**Pre-training:** web-scale raw knowledge and language structure acquisition.

**SFT:** teaching interaction formats, tasks, and system behaviors.

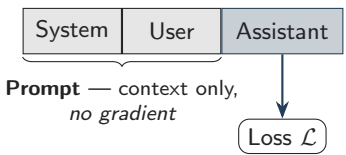
**RLHF/DPO:** micro-tuning behavior to meet safety standards and human feedback.

This 3-stage pipeline was introduced by Ouyang et al., *Training language models to follow instructions with human feedback* (InstructGPT, 2022).

# The Anatomy of an Instruction Example

## ChatML / OpenAI JSON template structure

```
[
  {"role": "system", "content": "You are a helpful coding assistant."},
  {"role": "user", "content": "How do I reverse a list in Python?"},
  {"role": "assistant", "content": "You can reverse a list using
     ‘list.reverse()‘ or slicing ‘[::-1]‘."}
]
```




***The Gold Target*** *— loss is calculated ONLY on the assistant response.*

**System:** defines boundaries/tone.

**User:** input instructions — part of the context, not the target.

**Assistant:** the training target — this is what the loss actually optimizes.

# Instruction Tuning Works: The FLAN Result

FLAN — Finetuned Language Models Are Zero-Shot Learners (Wei et al., 2022).

**Instruction tuning generalizes across tasks — not just new examples, but entirely new task types.**



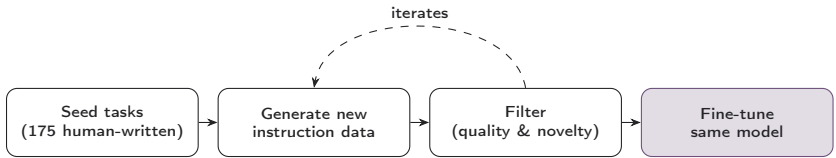

**Result:** Holding out entire task clusters (e.g., all NLI), zero-shot FLAN beats GPT-3 on **20 of 25** held-out types — even topping few-shot on several.

Ablations confirm real generalization: gains grow with task diversity and require natural-language phrasing.

# Bootstrapping Instructions from the Model Itself: Self-Instruct

Self-Instruct (Wang et al., 2023).

**Have the model generate its own training data, then filter.**




**Result:** +33.1% ROUGE-L over GPT-3 on Super-NaturalInstructions — within 5% of InstructGPT on novel tasks, minimal annotation.

Direct ancestor of the Llama 3 rejection-sampling pipeline (two slides ahead) — less refined.

# Where the Data Actually Comes From

**Both reports use mostly model-generated data — humans verify or select, not write from scratch.**

### DeepSeek-V3 — Model Generates, Humans Verify

**Reasoning** (math/code/logic): generated by **DeepSeek-R1**, an internal AI reasoning model, then rejection-sampled.

**Non-reasoning** (writing, role-play, QA): generated by **DeepSeek-V2.5**; humans *verify* correctness, don't write it.

### Llama 3 — Humans Write Prompts, Models Draft Responses

**Prompts:** collected via human annotation — real-world-style use cases.

**Responses:** mostly **rejection-sampled** ($K$=10–30, RM picks best) — not human-written.

Plus targeted **synthetic data** (code, long-context, tools) and a small hand-curated slice.

Both pipelines diverge from FLAN: humans supply prompts, models draft responses, then filter via sampling or verification.

# DeepSeek-V3: Quantity vs. Quality, by the Numbers

*A case study from one published model report — DeepSeek-V3.*

| Stage | Data | Compute |
|---|---|---|
| Pre-training | 14.8T tokens | 2,664K H800-hrs → \$5.33M |
| Context extension | YaRN, 2 phases × 1000 steps: 4K→32K→128K | 119K H800-hrs → \$0.24M |
| Post-training (SFT+RL) | 1.5M instruction instances | 5K H800-hrs → \$0.01M |

Context extension: no new data, just longer sequences — the model re-learns to attend over a longer window before it's taught to follow instructions. *Caveat: the post-training figure bundles SFT and RL; no SFT-only cost is published.*

DeepSeek-V3 at a glance — MoE, 671B total / 37B active params · 128K-token context · 128K BPE vocabulary

# What "Curated" Actually Means

*A case study from one published model report — Llama 3.*

### Pre-training Data

Scraped from the web — bulk, automatic, corpus-level

Dedup (URL / doc via MinHash / line)

↓

Heuristic filters (n-grams, dirty words, KL)

↓

fastText / RoBERTa classifiers

### SFT Data

Written/generated on purpose — per-example, model-judged

Rejection sampling ($K$=10–30)

↓

Clean & topic-tag

↓

Quality × difficulty score

↓

Semantic dedup

} ×6 rounds

Llama 3's own summary: *"more careful pre-processing and curation pipelines for pre-training data . . . more rigorous quality assurance and filtering approaches for post-training data."* §3.1 / §4.2.3.

# Quality Over Quantity: The LIMA Effect

**LIMA (Zhou et al., 2023): 1,000 curated examples outperform Alpaca's 52K and stay competitive with RLHF'd models like DaVinci003 and Bard.**

**Equal-or-Preferred Rate** — % of matchups where LIMA tied or won in blind comparisons.

| vs. baseline | DaVinci003 (RLHF) | Bard | Claude | GPT-4 |
|---|---|---|---|---|
| LIMA equal-or-preferred | **65%** | **58%** | 46% | 43% |

65B LLaMA, 1,000 examples, no RLHF.

# After SFT: Why RL Still Helps

Chu et al., 2025 — SFT and RL operate on different signal frequencies.

**SFT: Low-Frequency Structure**

Learns broad behavioral patterns (instruction-following across tasks).

But overfits to training data's high-frequency details.

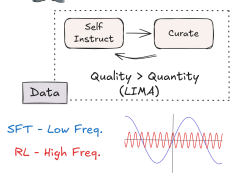
**RL: High-Frequency Adaptation**

Adapts to fine-grained variations (rule/visual tweaks within a task).

Explores solution space, handles distribution shifts.

**SFT is still necessary first** — it stabilizes output format for RL.

SFT captures **low-frequency patterns** (instruction-following behavior across tasks), but overfits to training data's high-frequency details. RL adapts to **high-frequency variation** (rule/visual tweaks within a task) — complementary, not contradictory. TailSFT (Malladi et al., 2026) filters well-fit sequences from SFT, improving pass@K for RL.


Self Instruct
Curate
Quality > Quantity (LIMA)
Data
SFT - Low Freq.
RL - High Freq.

# What Alignment SFT + RL Don't Give You

**Alignment SFT** (low-frequency)

- General instruction-following
- Recognize requests vs. continuations
- Safe, helpful persona

**RL / RLHF** (high-frequency)

- Preference alignment
- Refusal boundaries
- Response style polish

**Still missing:**

- *Your* output schema (JSON, XML, domain-specific DSL)
- *Your* vocabulary (legal terms, medical codes, internal jargon)
- *Your* edge cases (product-specific failure modes)
- Optimized accuracy on *your* task distribution

Recall the autonomous-driving example (slide 4): the general model *can* answer, but not in the structured risk/action schema the planner expects. That's a Domain SFT problem.

# Domain Adaptation Still Helps

**Key finding** *(Gururangan et al., 2020 — "Don't Stop Pretraining"):*
Even after broad pretraining, **continued domain- or task-specific training measurably improves downstream performance.**

| Adaptation Type | Improvement | What It Does |
|---|---|---|
| Domain-Adaptive Pretraining (DAPT) | +1–8 F1 | Continue pretraining on domain text |
| Task-Adaptive Pretraining (TAPT) | +1–3 F1 | Continue pretraining on task corpus |
| DAPT + TAPT | Best overall | Both stacked |

Tested across four domains: **biomedical**, **computer science**, **news**, **reviews**.

**Two benefits of Domain SFT:**

**Format** — your schema, output structure, vocabulary

**Accuracy** — measurable gains on domain tasks (the numbers above)

# Alternative 1: Prompt Engineering

**Idea:** Instead of retraining, steer the model via instructions and examples in the prompt.

| Aspect | Prompt Engineering | Domain SFT |
|---|---|---|
| Upfront Cost | $0 — immediate via API | High — data + training |
| Per-Request Cost | High — examples every call | Minimal — short prompts |
| Format Reliability | Can drift, model may ignore | Baked into weights |
| Update Speed | Instant | Requires retraining |

**When to use:**

Low volume, high variance tasks

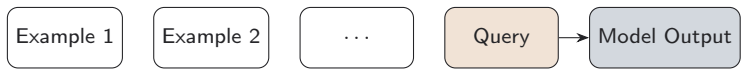
Rapid iteration / experimentation phase

No training infrastructure available

Prompt engineering is mostly an alternative to *domain* SFT (an already-instruction-following model). It can approximate alignment too, but fragile and paid per-request.

# Alternative 2: In-Context Learning (ICL)

**Idea:** Provide few-shot examples directly in the prompt—the model "learns" the task from context alone.




**Surprising finding** *(Min et al., 2022 — "Rethinking the Role of Demonstrations"):* Ground-truth labels in demos **barely matter**. What matters:

**Label space** — showing what outputs are possible

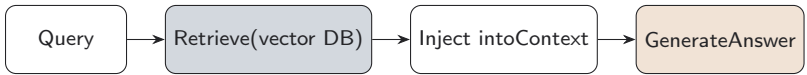
**Input distribution** — examples from the same domain

**Format** — how input/output pairs are structured

ICL is powerful but ephemeral: the "learning" vanishes when the context window closes. No persistent weight change.

# Alternative 3: Retrieval-Augmented Generation (RAG)

**Idea:** Automate what goes into the context by retrieving relevant documents at inference time.




**Key properties** *(Lewis et al., 2020):*

**Non-parametric memory:** knowledge lives in the index, not the weights

**Instant updates:** swap documents, knowledge changes immediately

**Provenance:** can cite which documents informed the answer

**Reduced hallucination:** model grounds responses in retrieved text

RAG excels at **knowledge**-intensive tasks (facts, QA). For **behavior** changes (output format, reasoning style), SFT is usually needed.

# RAG Variants: Active, Hierarchical, Graph-Based

The basic RAG pattern has many extensions:

| Variant | Key Idea | Best For |
|---|---|---|
| **Self-RAG** (Asai et al., 2023) | Model decides *when* to retrieve; critiques own outputs | Reducing unnecessary retrieval |
| **FLARE** (Jiang et al., 2023) | Retrieves *during* generation when confidence drops | Long-form generation |
| **RAPTOR** (Sarthi et al., 2024) | Hierarchical tree of summaries, not flat chunks | Multi-hop reasoning |
| **Graph RAG** (Edge et al., 2025) | Entity graph + community summaries | Complex entity relationships |

All variants share the core idea: inject external knowledge at inference time rather than baking it into weights. Choice depends on your retrieval patterns and reasoning complexity.

# Summary: When to Use What

| Criterion | Prompt Eng. | ICL | RAG | Domain SFT |
|---|---|---|---|---|
| Modifies weights? | No | No | No | **Yes** |
| Upfront cost | None | None | Index build | **High** |
| Per-request cost | High | High | Medium | **Low** |
| Update latency | Instant | Instant | Instant | Retrain |
| Best for | Simple steering | Format demos | **Knowledge** | **Behavior** |
| Reliability | Can drift | Ephemeral | Depends on retrieval | **Durable** |

**Rule of thumb:**

**Knowledge** (facts, documents, evolving info) $\rightarrow$ RAG

**Behavior** (format, style, reasoning patterns) $\rightarrow$ SFT

**Both** $\rightarrow$ SFT for behavior + RAG for knowledge

# Why Not Full Fine-Tuning?

**Full fine-tuning** = update every parameter in the model.

**Problems at scale:**

**Memory:** GPT-3 175B needs ∼700GB VRAM just for gradients + optimizer states

**Catastrophic forgetting:** model loses general capabilities when overtrained on narrow data

**Storage:** need a separate 175B-parameter checkpoint per task/customer

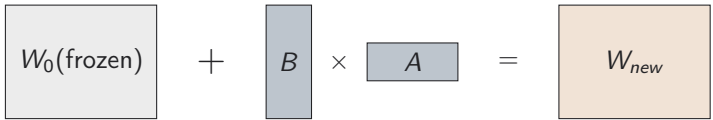
**Cost:** full training runs are expensive to repeat

**Solution: Parameter-Efficient Fine-Tuning (PEFT)**
Freeze most of the model; train only a small set of new or selected parameters.

PEFT is still SFT—same loss function, same training data. Only the *scope* of what gets updated changes.

# LoRA: Low-Rank Adaptation

**Core idea** *(Hu et al., 2021):* Weight updates are often low-rank—decompose $\Delta W$ into two small matrices.




$$W_{new} = W_0 + \frac{\alpha}{r} BA, \quad A \in \mathbb{R}^{r \times d_{in}}, \; B \in \mathbb{R}^{d_{out} \times r} \tag{1}$$

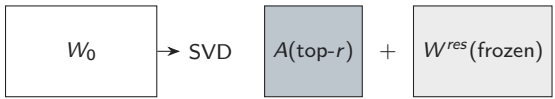
**Key properties:**

- **Trainable params:** $\sim$0.01% of full model (rank $r$ typically 4–64)
- **Memory:** 3$\times$ less GPU memory than full fine-tuning
- **Inference:** merge $BA$ into $W_0 \rightarrow$ **zero added latency**
- **Performance:** matches or exceeds full fine-tuning on many tasks

# LoRA Variants: PiSSA (SVD-Based Initialization)

**Problem with vanilla LoRA:** $A$ initialized with Gaussian noise, $B$ with zeros $\rightarrow$ slow convergence.

**PiSSA** *(Meng et al., 2025):* Initialize adapters with the **principal singular components** of $W_0$ itself.




**Results:**

Gemma-7B on GSM8K: PiSSA **77.7%** vs. LoRA 74.5% (+3.2%)

LLaMA-3-70B with 4-bit quant: QPiSSA **86.1%** vs. QLoRA 81.7% (+4.4%)

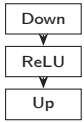
Initialization cost: seconds (fast SVD), negligible vs. training time

Same architecture as LoRA—drop-in replacement. The only change is *how* $A$ and $B$ are initialized.

# Other PEFT Families

## Adapters
*(Houlsby et al., 2019)*

Insert small bottleneck layers *inside* transformer blocks.




∼3% params per task.
Adds inference latency.

## Prefix-Tuning
*(Li & Liang, 2021)*

Prepend learnable "virtual tokens" to keys/values in every layer.




∼0.1% params.
No architecture change.

## Prompt Tuning
*(Lester et al., 2021)*

Learnable soft prompt at input layer only (not every layer).

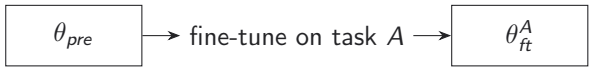
Simpler than prefix-tuning. Scales with model size: "closes the gap" at 10B+ params.

∼0.01% params.
Minimal overhead.

All methods freeze the base model and train only a small number of new parameters. Choice depends on your latency/memory constraints and how much expressiveness you need.

# Task Arithmetic: Combining Adapters

**Core idea** *(Ilharco et al., 2023):* A **task vector** = fine-tuned weights − pretrained weights.




$$\tau_A = \theta_{ft}^{A} - \theta_{pre}$$

**Operations on task vectors:**

**Negation:** $\theta_{pre} - \tau_A \rightarrow$ *reduces* performance on task $A$

**Addition:** $\theta_{pre} + \tau_A + \tau_B \rightarrow$ improves on *both* $A$ and $B$

**Analogy:** "$A$ is to $B$ as $C$ is to $D$" $\rightarrow$ combine $\tau_A$, $\tau_B$, $\tau_C$ to improve on $D$

**No retraining needed**—just arithmetic on saved weight deltas.

Works with LoRA too: each adapter is a task vector you can combine post-hoc.

# Summary: PEFT Method Comparison

| Method | Params | Memory | Inference | Where | Composable? |
|---|---|---|---|---|---|
| Full FT | 100% | Very high | None | All | No |
| **LoRA** | 0.01–0.1% | 3× lower | **None** (merge) | Attention | **Yes** |
| PiSSA | 0.01–0.1% | 3× lower | **None** | Attention | Yes |
| Adapters | 1–5% | Medium | Added | Each layer | Yes |
| Prefix-Tuning | 0.1% | Low | Minimal | K/V cache | Yes |
| Prompt Tuning | 0.01% | Very low | None | Input only | Yes |

**Practical defaults:**

**Start with LoRA** ($r$=8–32): best balance of efficiency and performance

**Try PiSSA** if LoRA converges slowly: same cost, faster training

**Use Adapters** if you need maximum expressiveness and can tolerate latency

**Task Arithmetic** to combine multiple LoRA/adapters without retraining

# Archived From Here Down

The following slides are reference material from earlier drafts.

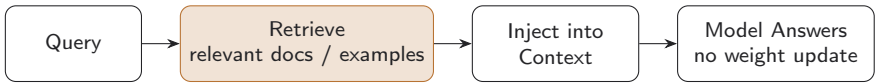
Not part of the main presentation.

# Prompt Engineering vs. Domain SFT

| Metric | Prompt Engineering (Few-Shot) | Supervised Fine-Tuning (SFT) |
|---|---|---|
| Upfront Cost | $0 — immediate setup via API | High — data collection + GPU rent |
| Cost Per Request | High — paying for examples every time | Minimal — short, clean prompts |
| Format Reliability | Weak — model can drift or forget rules | Hardcoded — baked into model weights |

Mostly a choice against *domain* SFT specifically (recall slide 3) — an already instruction-following model can often be steered by examples instead of retrained. Prompt engineering can also approximate alignment itself, but you pay for it on every request, and it's fragile; SFT bakes the behavior into the weights once.

# A Third Axis: In-Context Learning & RAG




**In-context learning:** few-shot examples or instructions placed directly in the prompt — no training, forgotten the moment the request ends.

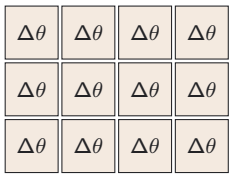
**RAG (Retrieval-Augmented Generation):** automates *what* goes into that context, pulling relevant passages from a knowledge base instead of hand-picking examples.

RAG is a technique that allows LLMs to retrieve relevant information from a knowledge base.

SFT bakes behavior into the weights — durable, cheap per request, but paid for upfront. RAG/ICL instead change what's in the context — cheap to update, but paid on every request. Rule of thumb: RAG for *knowledge*, SFT for *behavior*.

# Full Fine-Tuning vs. PEFT (LoRA)

## Full Fine-Tuning (FT)



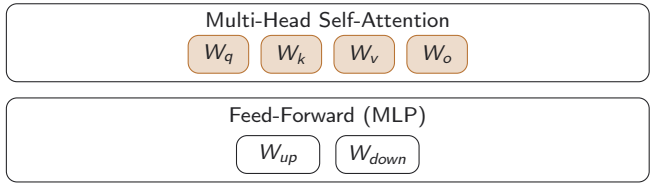

Every parameter updates. High risk of catastrophic forgetting; massive VRAM for gradients + optimizer states.

## PEFT — Variant, Not Alternative




Still SFT — same masked CE loss, same data. Only a tiny fraction of parameters (the LoRA matrices) actually train; the base model stays frozen.

# LoRA Inside the Transformer




$$W_{\text{new}} = W_0 + \frac{\alpha}{r}(B \times A), \quad A \in \mathbb{R}^{r \times d}, \; B \in \mathbb{R}^{d \times r} \tag{2}$$

**Targeted, not global:** each adapted matrix gets its own low-rank pair (A, B) — LoRA is applied per-weight-matrix, not once to the whole model.

**Attention first:** $W_q/W_k/W_v/W_o$ are the standard targets — they control *what the model attends to*.

**MLP is optional:** larger, and holds more world knowledge — costs more to adapt but helps when the task needs new knowledge pathways.

# Retrieval-Augmented Generation (RAG)

RAG (Lewis et al., 2020) shifts the focus entirely away from parameter adaptation. Rather than embedding domain knowledge into the model's internal weights via gradient updates, RAG dynamically fetches relevant text chunks from an external vector database at inference time and injects them directly into the prompt context.

| Feature | PEFT | RAG |
|---|---|---|
| Core Mechanism | Modifies internal neural weights. | Modifies the input prompt context. |
| Primary Strength | Teaching new formats, tone, syntax, or distinct reasoning patterns. | Injecting massive datasets of factual, proprietary, or highly dynamic knowledge. |
| Updating Knowledge | Requires re-running the optimization and training process. | Instantaneous database updates without touching the model. |
| Hallucination Risk | High; the model relies on parametric memory and may confidently invent facts. | Low; the model is explicitly grounded by the retrieved external context. |

# Vector Databases & Similarity Search

**Common Similarity Measures:**

- Euclidean Distance
- Cosine Similarity
- Dot Product

# Vector Databases & Similarity Search

**Common Similarity Measures:**

- Euclidean Distance
- Cosine Similarity
- Dot Product

**Indexing Methods (Sacrifice Accuracy for Speed):**

| Method | Core Idea | How it Works | Strengths | Best For |
|---|---|---|---|---|
| **Hierarchical Navigable Small World (HNSW)** | Graph | Navigates a multi-layered graph to find neighbors. | Fast & Accurate | Performance and accuracy-critical use cases. A common default. |
| **Inverted File (IVF)** | Clustering | Searches only within the most relevant vector clusters. | Memory Efficient, Scalable | Very large datasets where memory is a primary constraint. |
| **Locality-Sensitive Hashing (LSH)** | Hashing | Hashes similar vectors into the same "bucket" for lookup. | Massively Scalable | Huge datasets where speed is prioritized over perfect accuracy. |